UNIVERSITI PENDIDIKAN SULTAN IDRIS

اونيۏرسيتي ڤنديديقن سلطان ادريس

SULTAN IDRIS EDUCATION UNIVERSITY

Software Verification and Validation (DES3103)

Course Learning Outline

1. Briefing Session (Ice Breaking via Whiteboard, Class rules & Pro forma): **M. Teams**
2. Introduction to Software Verification & Validation I & II (V&V)
3. Managing V&V with SQA Integration
4. Quality metrics or attributes in V&V

WEEK 6-MID-TERM TEST (QUIZ): Topic 1 – 3-(CLO1:C2) – 20 MARKS

WEEK 7-PHASE 1: ASSIGNMENT 1-ONLINE PROJECT-BASED LEARNING (OPBL)-(CLO3:A5) - 20 MARKS: M. Teams

WEEK 9-PHASE 1: PRESENTATION & REFLECTION ON PHASE 1: M. Teams

5. Software Reviews – Will be covered on Assignments 1 & 2
6. Quality Control: Software Testing - Will be covered in Assignments 1 & 2
7. Human-Computer Interface Testing - Will be covered in Assignments 1 & 2
8. Manual Testing and Automation Testing: Will be covered in Final Assessment

WEEK 11-PHASE 2: ASSIGNMENT 2-ONLINE PROJECT-BASED LEARNING (OPBL)-(CLO2:P4) - 20 MARKS: M. Teams
WEEK 13-PHASE 2: PRESENTATION & REFLECTION ON PHASE 2: M. Teams

9. Quality Maturity Models
10. Software Problem Analysis and Reporting
11. Quality Culture in doing V&V.
12. Safety and Security in V&V

WEEK 14: REFLECTION ON IMPLEMENTATION OPBL (SOFTWARE TESTERS) IN THE V&V COURSE: e-FORUM

WEEK 16-PHASE 3: FINAL ASSESSMENT – INDIVIDUAL SPECIAL PROJECT BASED ON PHASE 2 (CLO4:C5) (2 WEEKS) - 40 MARKS : e-ASSESSMENT

TOPIC 1

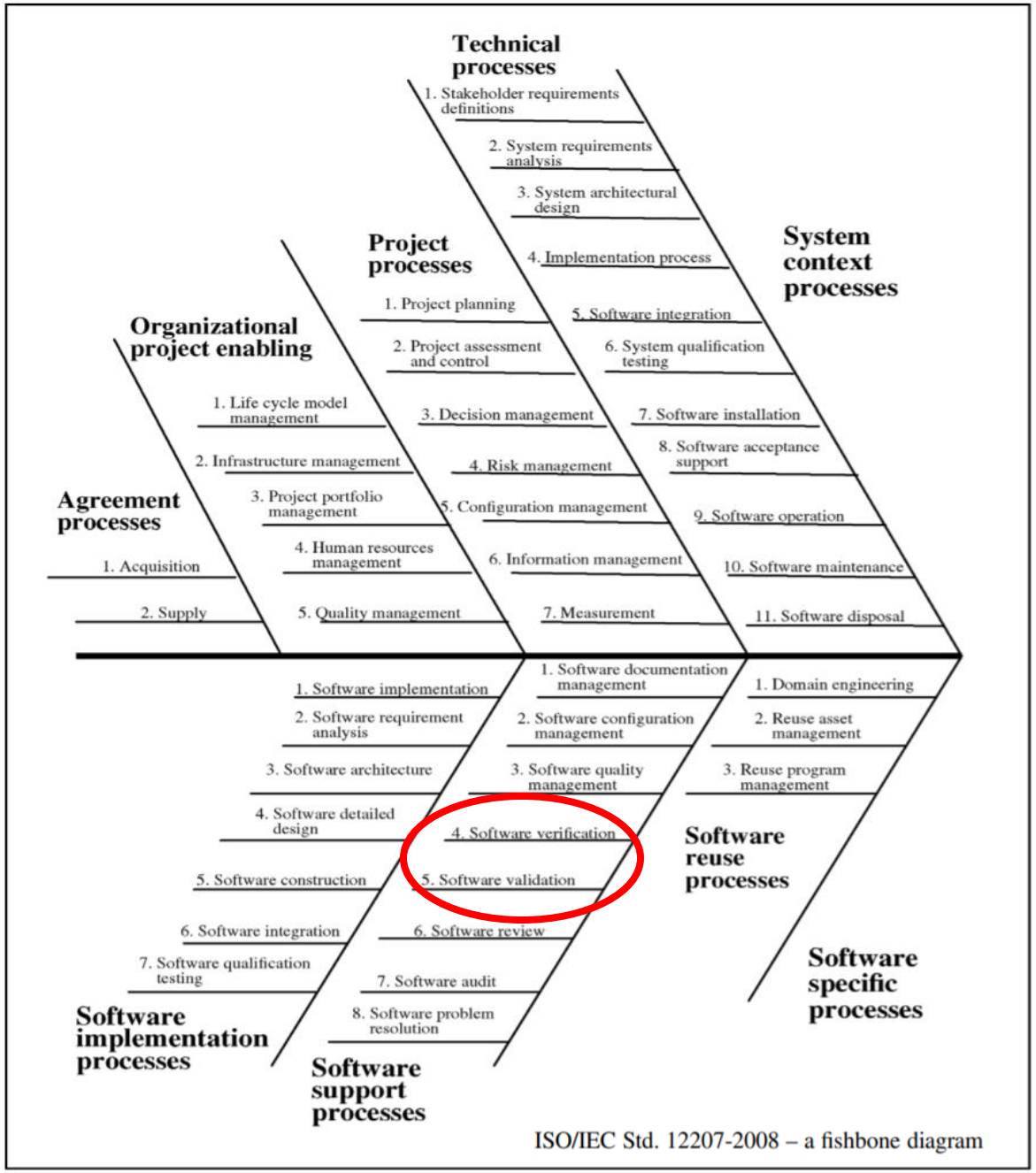
UNIVERSITI PENDIDIKAN SULTAN IDRIS
اونيۏرسيتي ڤنديديقن سلطان ادريس
SULTAN IDRIS EDUCATION UNIVERSITY

Introduction to Software Verification & Validation (V&V)

UNIVERSITI PENDIDIKAN SULTAN IDRIS
اونيۏرسيتي ڤنديديقن سلطان ادريس
SULTAN IDRIS EDUCATION UNIVERSITY

Introduction to Software Verification & Validation (V&V)

- Introduction to Software Verification & Validation (V&V)
  - What you should know about V&V
    - Verification vs. Validation vs. Testing
    - Static Testing vs. Dynamic Testing



ISO/IEC Std. 12207-2008 – a fishbone diagram

Who should be involved?

Software quality assurance (SQA) team - An independent group of individual whose primary responsibility to ensure the delivered product is what the client needs and that the product has been built correctly in every way.

Tester - A person who tries to find out all possible errors / bugs in the system with the help of various inputs to it.

Skills required:

Analytical skills

Communication skills

Time management & organization skills

Great attitudes

Passion

(http://www.guru99.com/software-testing-career-complete-guide.html)

What they do?

V&V is carried out in parallel with the software/system development process.

V&V activities include:

Traceability analysis

Evaluation

Review

Inspection

Assessment

Testing

Refer to *IEEE 1012 - Standard for Software Verification and Validation*[2] for a comprehensive description of V&V life cycle activities.

How Much V&V is Required?

- The level of effort applied to V&V is a function of the criticality of the software or systems product. That is, the risks involved if the system fails.
  - At one end of the scale the software controlling the shutdown of a nuclear reactor **will likely be thoroughly verified and validated** by an independent organization.
  - At the other end of the scale, a website providing a company brochure **will likely have no formal verification and validation applied.**

How Much V&V Do You Need?

The level of effort you expend on V&V should be commensurate with the size and criticality of your application. If you are uncertain as to how much V&V you need on your project take **the Socratic approach** and ask yourself these hard questions:

i. **Is the system safety critical? YES**
   Could the failure of the software cause death, personal injury, mission failure or financial or catastrophic equipment loss or damage?

ii. **Is the target technology immature or high-risk? YES**
   Is this the first time this technology has been deployed in this type of application?

iii. **Is the project monolithic? YES**
   Is this a monolithic development with multiple development teams?
   https://www.techtarget.com/whatis/definition/monolithic-architecture

iv. **Will the business tolerate a high-risk project? NO**
   - ✓ If insufficient funds and resources are available for a full-service V&V program is the business prepared to accept the increased risk of project failure?
   - ✓ If the answer is **YES** to any of questions 1 to 3 or **NO** to question 4 your project needs verification and validation services.
     https://www.chambers.com.au/glossary/work_breakdown_structure_services.php

Introduction to Software Verification & Validation (V&V)

The importance of conducting proper software (V&V)

BENEFITS OF VERIFICATION AND VALIDATION

- Facilitate early detection and correction of software anomalies.
- Enhance management insight into process and product risk.
- Support the software life cycle processes to ensure conformance to program performance, schedule, and budget.
- Provide an early assessment of software and system performance.
- Improve the software development and maintenance processes.
- Support the process improvement for software system integration.

The Benefits of V&V Best Practice

Learning outcomes:

- Analyze the differences between software verification & validation
- Work collaboratively in a team

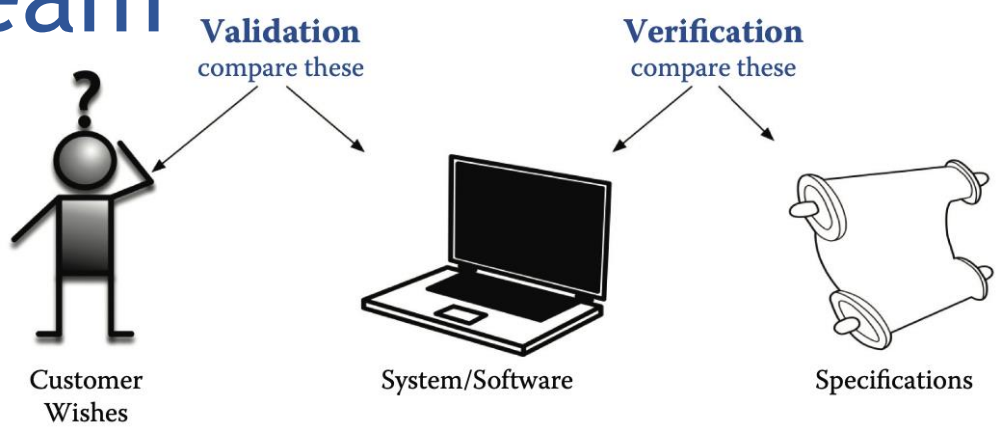


FIGURE 9.2 Two kinds of correctness and how they are ensured.

Introduction to Software Verification & Validation (V&V)

Verification vs. Validation
(Schach, 2011, page 167)

- **Verification – the** process of determining whether a workflow has been correctly carried out / mathematical techniques for showing that a product is correct (correctness proof) / denotes all non-execution-based techniques

- **Validation** – the intensive evaluation process that takes place just before the product is delivered to the client to determine whether the product as a whole satisfies it specifications.

Introduction to Software Verification & Validation (V&V)

Verification vs. Validation vs. Testing

- **Verification** - prefer to use term 'testing' rather than 'V&V' because of the undesirable implication:
  - Word 'verification' has another meaning within the context of testing
    - V&V implies that the process of checking a workflow
    - V&V can wait the end of that workflow

Verification vs. Validation vs. Testing

- **Testing** – execution of the software with actual test data (sometimes called as the dynamic software testing to distinguish another 'testing' called static 'testing', the static analysis)

(Source: Gustafson, 2002)

Verification vs. Validation vs. Testing

How about others' opinion?

Verification vs. Validation vs. Testing

Otera (2012) decided to follow the IEEE (1990, p. 76) who defines **testing** as:

- The process of operating a system or component under specified conditions, observing or recording the results, and making an evaluation of some aspect of the system of component.

- The process of analyzing a software item to detect the differences between existing and required condition, such as bugs, and to evaluate the feature of the software item.

Verification vs. Validation vs. Testing

International Software Testing Quality Board (ISTQB) - Standard Glossary of Terms Used in Software Testing in 2015 for expert level (Test Manager) defines :

- **Testing** – the process consisting of all lifecycle activities, both **static and dynamic**, concerned with planning, preparation and evaluation of software products and related work products to determine that they satisfy specified requirements, to demonstrate that they are fit for purpose and to detect defects.

- **Verification (Ref: ISO 9000)** – confirmation by examination and through provision of objective evidence that specific requirements have been fulfilled.

- **Validation (Ref: ISO 9000)** – confirmation by examination and through provision of objective evidence that the requirements for a specific intended use or application have been fulfilled.

Static testing vs. Dynamic testing

- **Static software testing** – to improve the quality of the software products by finding errors in early stages of software development life cycle (SDLC).

- Static testing is the VERIFICATION part of the V&V
- Non-execution based testing

- Static testing techniques:

  - Informal reviews
  - Walkthrough
  - Technical reviews
  - Inspection

- Static Code Analysis = analysing the source code to identify problems

Introduction to Software Verification & Validation (V&V)

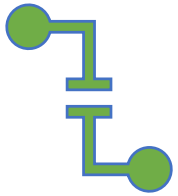
Static testing vs. Dynamic testing

- **Dynamic software testing** – an actual execution of the software with actual test data.
  - Dynamic testing is the VALIDATION part of the V&V
  - An execution-based testing
  - Must be held before handover to client.
  - Should be done by someone else not by the programmer himself / herself
  - Dynamic testing techniques:
    - Usability testing
    - User acceptance test
    - Black box testing (functional)
    - White box testing (structural-data)

Dynamic testing - Why testing should not be conducted by programmer?

Changes of programmer to use test data that will highlight his/her fault is considerably lower because of protective of his/her own codes' attitude. Need to find some one with major motivation were truly destructive.

A programmer may have misunderstood some aspect of the design or specification



Nevertheless, debugging (finding the cause of failure and correcting the fault) is best done by the original programmer as the person most familiar with the code. However, this does not mean a programmer should not test his or her own code.

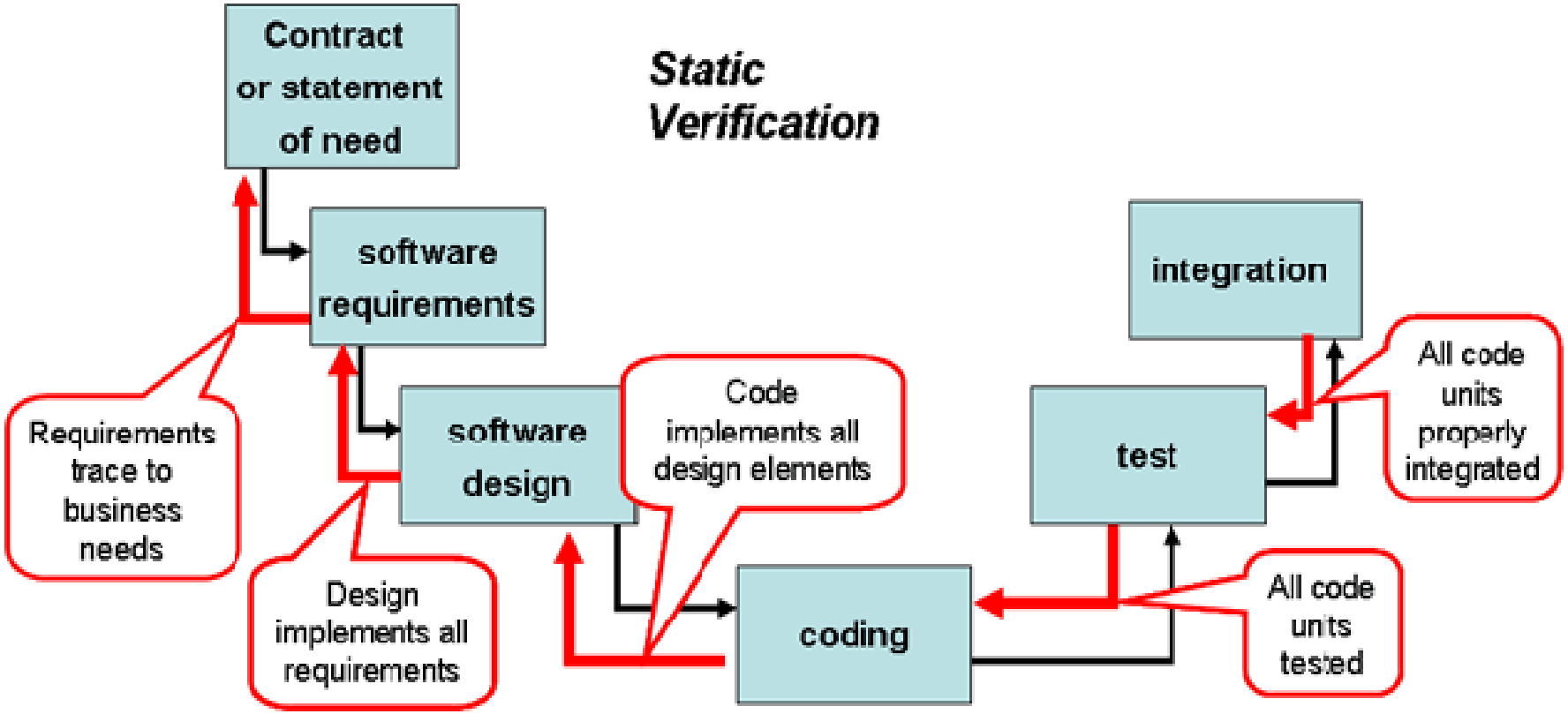
Introduction to Software Verification & Validation (V&V)

What Is Verification?

Verification is achieved through:

Static analysis of documentation (refer figure: Static Verification)

- Evaluation of work products in the context of reviews

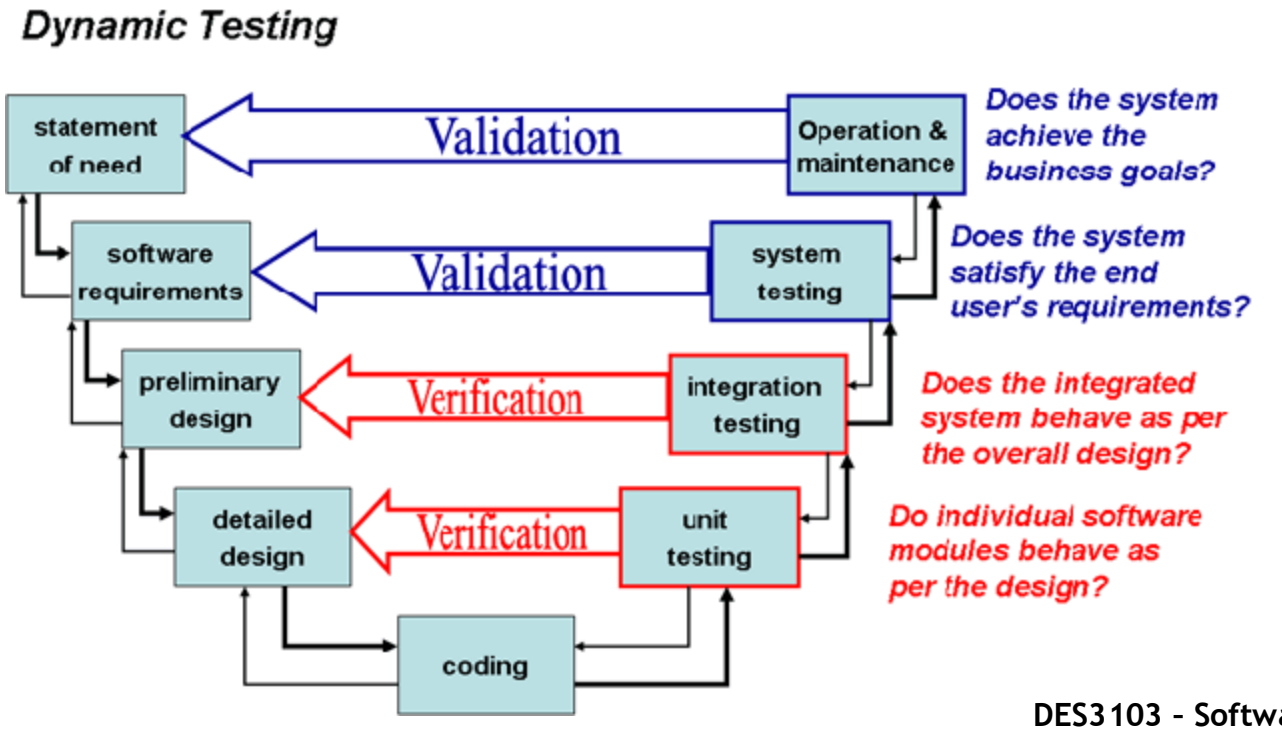


Introduction to Software Verification & Validation (V&V)

What Is Validation?

Validation demonstrates that a software or systems product is fit for purpose. That is, it satisfies all the customer's stated an implied needs.

Validation can be performed progressively throughout the development life cycle. For example, written user requirements can be validated by creating a model or prototype and asking the user to confirm (or validate) that the demonstrated functionality meets their needs. **System testing is a major validation** event where a system is validated against the user's statement of requirement. It aims to show that all faults which could degrade system performance have been removed before the system is operated. Validation is not complete however until the end user formally agrees that the operational system is fit for purpose.



Introduction to Software Verification & Validation (V&V)

The **verifier** establishes that a software product faithfully implements all the requirements documented in a **software requirements specification (SRS).**

Validator establishes that the SRS is a true reflection of the user's needs. For example, he might use a prototype to encourage users to think more deeply about their needs and discover that some of the requirements were incorrectly stated in the first place.

The **verifier** makes sure that the specifications are followed to the letter - was the Wright Flier constructed as per its specification?

The **validator** therefore establishes that the specifications correctly describe a system that is fit for its intended purpose - will the Wright Flier actually fly?

Introduction to Software Verification & Validation (V&V)

Verification and validation is a systems engineering discipline that determines if work products comply with their specifications and are fit for their intended use.

More specifically:

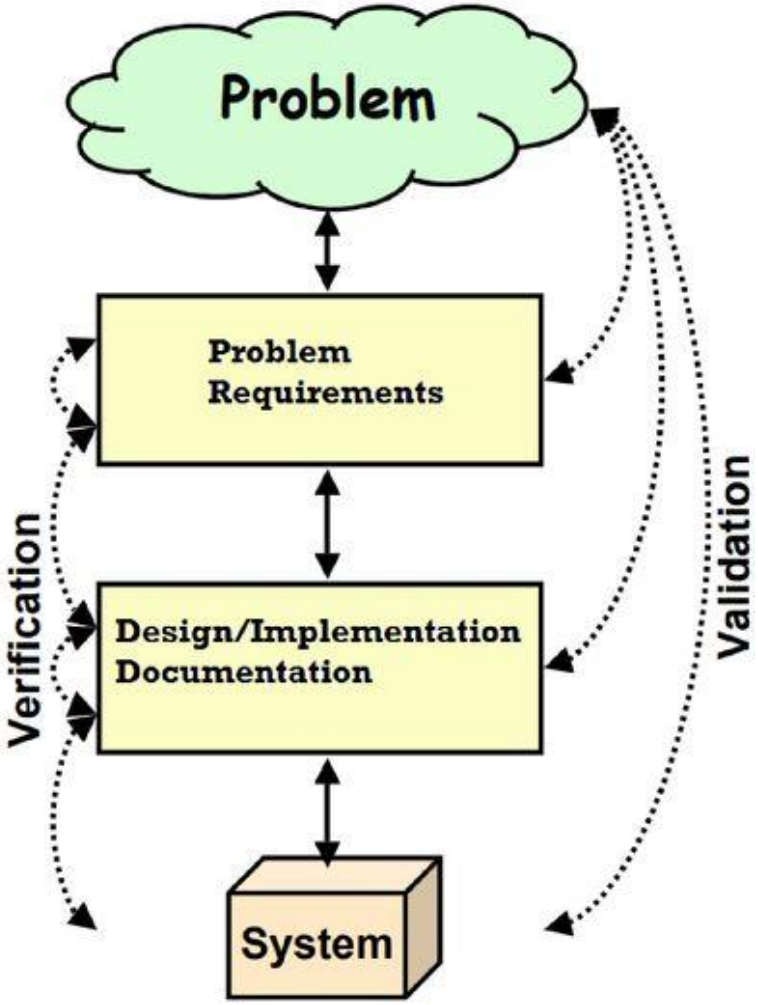
Verification establishes the truth of correspondence between a work product and its specification (from the Latin *veritas*, "truth").
It answers the question: "**Are we building the product right**?"

Validation establishes the fitness of a software product for its operational mission (from the Latin *valere*, "to be worth").
It answers the question: "**Are we building the right product**?"

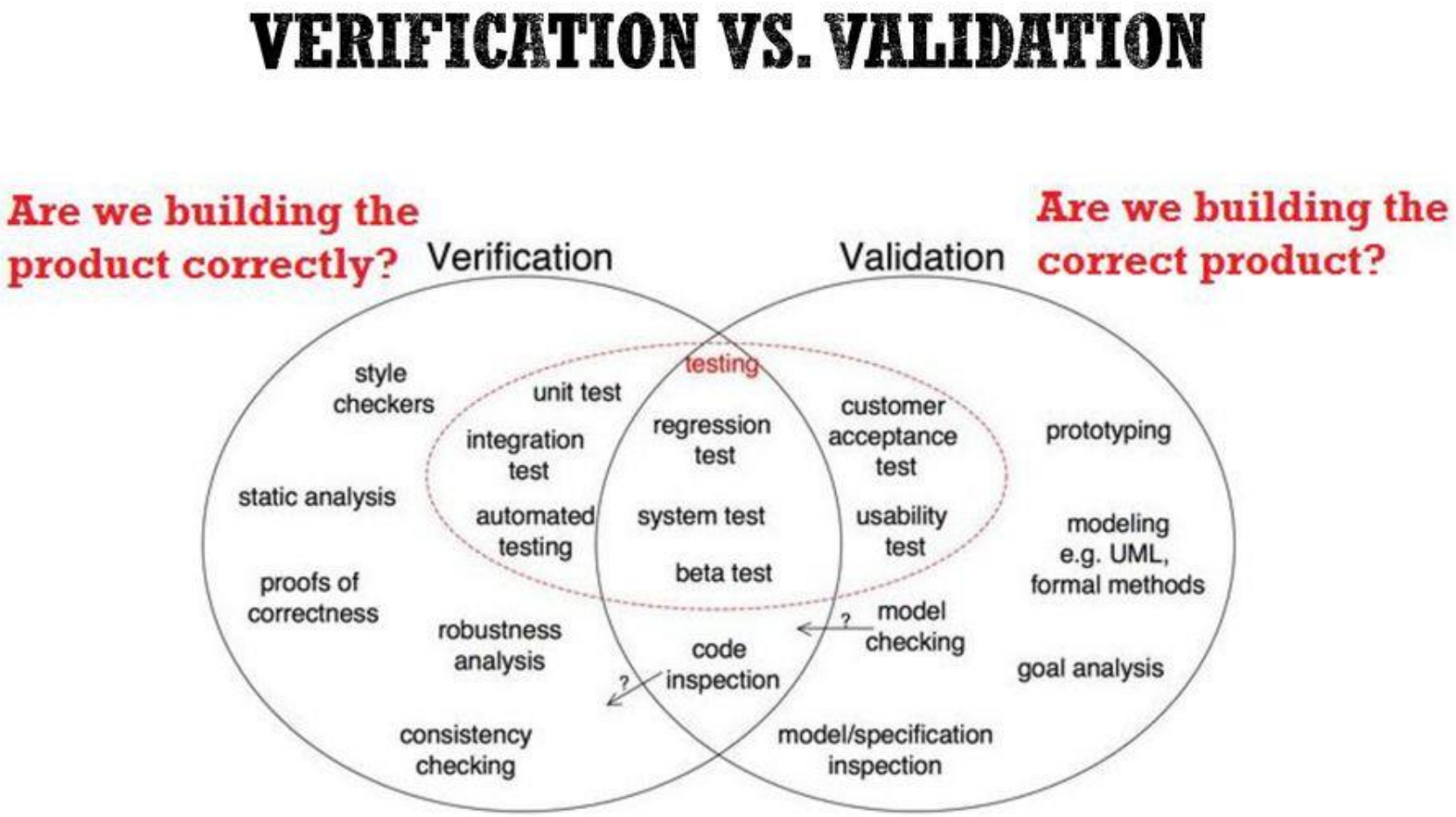
Introduction to Software Verification & Validation (V&V)

VERIFICATION AND VALIDATION

- **Verification testing:**
  - **"Are we building the product correctly?"**
    - **Does the design meet the reqs?**
    - **Does the implementation meet the reqs and match the models?**
    - **The software should conform to its specifications**
      - Functional/non-functional requirements
- **Validation testing:**
  - **"Are we building the correct product?"**
    - **Do our problem requirements accurately capture the real problem?**
    - **Did we account for the needs of all stakeholders?**
    - **The software should do what the users really require.**



Introduction to Software Verification & Validation (V&V)



References

- https://sqa.jdn.gov.my/index.php/ms/senarai-dokumentasi
- Basu, A. (2015). Software Quality Assurance, Testing and Metrics, PHI Learning Klosterboer.
- Claude, Y. Laporte & Alain April (2018). Software Quality Assurance, the IEEE Computer Society, Inc, IEEE PRESS, WILEY.
- David, B. Sayre (2017). A Runtime Verification and Validation Framework for Self-Adaptive Software, Nova Southeastern University.
- Danial, G. (2018). Software Quality : Concepts and Practice, the IEEE Computer Society, Inc, IEEE PRESS, WILEY.
- JoonSeub, C. (2017). Verification and Validation of Automotive Embedded Software, Smart Media.
- Mistrik, I, Soley, M. R., Ali, N., Grundy, J., & Tekinerdogan, B. (2015). Software Quality Assurance: In Large Scale and Complex Software-intensive Systems.
- Tiziana, M. & Bernhard, S. (2018). Leveraging Applications of Formal Methods, Verification and Validation. Verification: 8th International, Springer.

END

UNIVERSITI PENDIDIKAN SULTAN IDRIS
اونيۏرسيتي ڤنديديقن سلطان ادريس
SULTAN IDRIS EDUCATION UNIVERSITY

Thank You…
Terima kasih..